

Problem Statement :

Design, implement, and evaluate an Intelligent Student Academic Performance and At-Risk Prediction System using machine learning techniques. An educational institution wants to develop a machine learning-based prototype that can analyse student academic information and identify students who may require additional academic support. The system will use a suitable student-performance dataset containing attributes such as attendance percentage, previous examination marks, study hours, assignment completion, internal assessment score, participation level, internet access, parental education, learning resources, and other relevant academic factors. The proposed system should analyse student data and classify students into suitable performance or academic-risk categories such as High Performing, Average Performing, and At Risk. The selected dataset and target variable should be clearly defined by the student.

Machine Learning Workflow: Student Dataset → Data Preprocessing → Statistical Analysis → Feature Selection → Model Development → Prediction → Model Evaluation → Academic Risk Classification Students

1. Introduction

Student academic performance is influenced by several factors such as attendance, previous examination marks, study hours, assignment completion, internal assessment scores, classroom participation, internet access, parental education, and availability of learning resources.

The proposed system, **Intelligent Student Academic Performance and At-Risk Prediction System Using Machine Learning**, analyses these factors and predicts the academic performance category of a student. The system classifies students into **High Performing, Average Performing, and At Risk** categories.

The system follows a machine learning workflow consisting of data preprocessing, statistical analysis, feature selection, model development, prediction, model evaluation, and academic risk classification.

2. Problem Statement

An educational institution wants to develop a machine learning-based prototype that can analyse student academic information and identify students who may require additional academic support.

The proposed system uses student-performance data containing academic and learning-related attributes. A machine learning model is trained using these attributes to classify students into suitable performance or academic-risk categories.

3. Objectives

1. To analyse student academic and learning-related information.
2. To preprocess and prepare the student dataset for machine learning.

3. To identify important features affecting academic performance.
4. To develop a Decision Tree classification model.
5. To predict whether students are High Performing, Average Performing, or At Risk.
6. To evaluate the performance of the machine learning model.
7. To provide early identification of students who may need academic support.

4. Dataset Description

A Student Academic Performance Dataset is used for this project. The dataset contains academic, behavioural, and learning-related attributes.

Input Features

- Attendance Percentage
- Previous Examination Marks
- Study Hours
- Assignment Completion
- Internal Assessment Score
- Participation Level
- Internet Access
- Parental Education
- Learning Resources

Target Variable

The target variable is **Performance_Risk**.

It consists of three categories:

- **High Performing**
- **Average Performing**
- **At Risk**

5. Hypothesis

Students with higher attendance, examination marks, study hours, assignment completion, and internal assessment scores are expected to have better academic performance.

Students with lower attendance, poor examination marks, fewer study hours, and low academic participation are more likely to be classified as **At Risk**.

6. Machine Learning Workflow

The proposed system follows the workflow:

Student Dataset → Data Preprocessing → Statistical Analysis → Feature Selection → Model Development → Prediction → Model Evaluation → Academic Risk Classification

7. Pseudocode

Algorithm: Student Academic Performance Prediction

1. Start.
2. Load the student dataset.
3. Display the dataset.
4. Check for missing values.
5. Handle missing values if present.
6. Encode categorical attributes.
7. Separate input features and target variable.
8. Split the dataset into training and testing sets.
9. Perform statistical analysis using Pandas.
10. Select relevant features.
11. Create a Decision Tree classifier.
12. Train the classifier using training data.
13. Predict the performance category for test data.
14. Calculate accuracy, precision, recall, and F1-score.
15. Generate the confusion matrix.
16. Classify students as High Performing, Average Performing, or At Risk.
17. Display the results.
18. Stop.

8. Data Preprocessing

Data preprocessing is performed before training the machine learning model.

The following operations are carried out:

8.1 Handling Missing Values

The dataset is checked for missing or null values. Missing numerical values can be replaced using the mean or median, while missing categorical values can be replaced using the mode.

8.2 Encoding Categorical Data

Categorical attributes such as Participation, Internet Access, Parental Education, and Learning Resources are converted into numerical values so that they can be processed by the machine learning algorithm.

8.3 Feature and Target Separation

The student-related attributes are considered as input features, while **Performance_Risk** is considered the target variable.

8.4 Train-Test Split

The dataset is divided into training and testing sets. The training data is used to build the model, while the testing data is used to evaluate its prediction performance.

9. Statistical Analysis Using Pandas

Pandas is used to load, inspect, filter, group, and analyse the student dataset.

Important operations include:

- Displaying the first few records.
- Finding dataset dimensions.
- Checking data types.
- Checking missing values.
- Calculating statistical measures.
- Filtering students based on attendance or marks.
- Grouping students based on performance category.
- Sorting students based on marks or attendance.

10. Feature Selection

Feature selection identifies the attributes that are useful for predicting academic performance.

Important features considered in this project include:

- Attendance
- Previous Examination Marks
- Study Hours
- Assignment Completion
- Internal Assessment Score
- Participation Level

These features can strongly contribute to distinguishing high-performing, average-performing, and at-risk students.

11. Decision Tree Design

A Decision Tree is selected as the primary classification algorithm because it is simple to understand and suitable for classification problems.

The tree consists of:

- **Root Node** – Most useful attribute for the first decision.
- **Internal Nodes** – Additional feature-based decisions.
- **Branches** – Possible outcomes of a decision.
- **Leaf Nodes** – Final performance categories.

For example:

Attendance → Previous Marks → Study Hours → Performance Risk

The final leaf node produces one of the three classifications:

High Performing / Average Performing / At Risk

12. Heuristic Attribute Selection

The Decision Tree uses a heuristic measure to select the best attribute for splitting the data.

Entropy

Entropy measures the impurity or uncertainty in the dataset.

$$\text{Entropy}(S) = -\sum p_i \log_2(p_i)$$

where p_i represents the proportion of samples belonging to a particular class.

Information Gain

Information Gain measures the reduction in uncertainty after splitting the dataset.

$$\text{Information Gain} = \text{Entropy}(\text{Parent}) - \text{Weighted Entropy}(\text{Children})$$

The attribute with higher information gain can be selected as a better splitting attribute.

13. Decision Tree Training

The Decision Tree is trained using the preprocessed student dataset.

The model learns relationships between student features and the target performance category.

After training, the model can predict the performance category of previously unseen students.

14. Python Implementation

```
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import train_test_split
```

```
from sklearn.metrics import accuracy_score
```

```
# Attendance, Previous Marks, Study Hours
```

```
X = [  
    [95, 90, 5],  
    [90, 85, 4],  
    [85, 80, 4],  
    [75, 70, 3],  
    [70, 65, 2],  
    [65, 60, 2],  
    [50, 45, 1],  
    [45, 40, 1],  
    [40, 35, 1],  
    [92, 88, 5]  
]
```

```
# 2 = High, 1 = Average, 0 = At Risk
```

```
y = [2, 2, 2, 1, 1, 1, 0, 0, 0, 2]
```

```
# Split data
```

```
X_train, X_test, y_train, y_test = train_test_split(  
    X, y, test_size=0.2, random_state=42  
)
```

```
# Decision Tree
```

```
model = DecisionTreeClassifier(random_state=42)
```

```
model.fit(X_train, y_train)
```

```
# Prediction
```

```

y_pred = model.predict(X_test)

print("Actual :", y_test)
print("Predicted :", y_pred)
print("Accuracy :", accuracy_score(y_test, y_pred) * 100, "%")

# New student
new_student = [[88, 82, 4]]
result = model.predict(new_student)[0]

categories = {
    2: "High Performing",
    1: "Average Performing",
    0: "At Risk"
}

print("New Student:", categories[result])

```

16. Model Evaluation

The Decision Tree model is evaluated using the following metrics.

Accuracy

Accuracy represents the proportion of correctly classified students.

Accuracy = Correct Predictions / Total Predictions

Precision

Precision measures how many students predicted as belonging to a particular category actually belong to that category.

Precision = TP / (TP + FP)

Recall

Recall measures how many actual students in a category are correctly identified.

Recall = TP / (TP + FN)

F1-Score

F1-score provides a balance between precision and recall.

$$\mathbf{F1} = 2 \times (\mathbf{Precision} \times \mathbf{Recall}) / (\mathbf{Precision} + \mathbf{Recall})$$

Confusion Matrix

A confusion matrix shows the correct and incorrect predictions for each performance category.

17. Academic Risk Classification

The system produces three categories.

High Performing

Students with strong academic indicators such as high attendance, good marks, sufficient study hours, and good assignment completion.

Average Performing

Students with moderate academic performance who may benefit from regular monitoring.

At Risk

Students showing weaker academic indicators and who may require additional academic support.

18. Neural Network Concepts

18.1 Perceptron

A perceptron is a basic artificial neuron used for classification. It receives input features, applies weights and a bias, and produces an output using an activation function.

18.2 Multilayer Perceptron

A Multilayer Perceptron (MLP) consists of an input layer, one or more hidden layers, and an output layer. It can learn complex relationships between student attributes and academic performance.

18.3 Backpropagation

Backpropagation is a learning algorithm used to update neural-network weights. The prediction error is propagated backward through the network, and the weights are adjusted to reduce the error.

For this project, the Decision Tree is used as the main implementation, while Perceptron, MLP, and backpropagation are analysed as alternative learning approaches.

19. Genetic Algorithm

A Genetic Algorithm (GA) is an evolutionary optimization technique inspired by natural selection.

The main steps are:

1. **Population Initialization** – Generate possible solutions.
2. **Fitness Evaluation** – Measure the quality of each solution.
3. **Selection** – Select better solutions.
4. **Crossover** – Combine selected solutions.
5. **Mutation** – Introduce small random changes.
6. **Termination** – Stop when a suitable solution is obtained.

Role in Hypothesis-Space Search

A Genetic Algorithm can search a large hypothesis space by representing possible machine learning models or feature combinations as individuals. The fitness function evaluates their performance, allowing better solutions to survive and generate new candidates.

In this project, GA can conceptually be used to identify useful feature combinations or optimize model parameters.

20. Menu-Driven Functionality

A menu-driven interface can make the system easier to use.

Example:

===== STUDENT PERFORMANCE PREDICTION =====

1. Display Dataset
2. Statistical Analysis
3. Train Decision Tree
4. Predict Student Risk
5. Evaluate Model
6. Exit

Enter your choice:

The user can select an operation and obtain the corresponding result.

21. Prediction of a New Student

A new student's details can be entered into the system.

Example:

Attendance = 85%

Previous Marks = 82

Study Hours = 4

Assignment Completion = 90%

Internal Score = 84

Participation = High

Internet Access = Yes

The trained Decision Tree processes the input and predicts the student's academic performance category.

Example output:

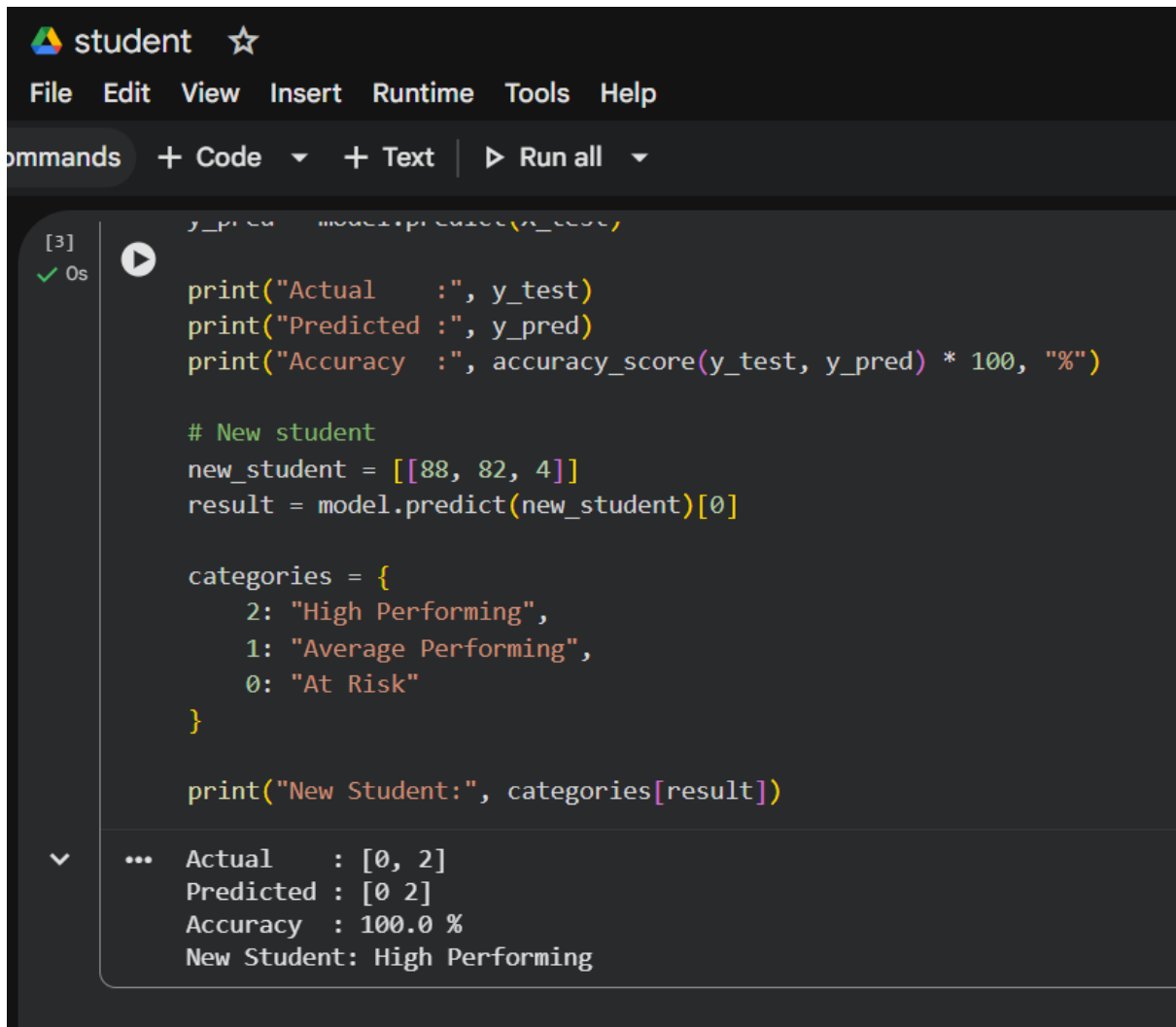
Predicted Category: High Performing

22. Output Results

After training and testing the Decision Tree, the system should display:

- Model accuracy
- Precision
- Recall
- F1-score
- Confusion matrix
- Predicted academic risk category

The actual numerical performance values should be obtained by running the Python program on the final dataset.



```
student ☆
File Edit View Insert Runtime Tools Help
Commands + Code + Text | ▶ Run all

[3]
✓ 0s
print("Actual      :", y_test)
print("Predicted   :", y_pred)
print("Accuracy    :", accuracy_score(y_test, y_pred) * 100, "%")

# New student
new_student = [[88, 82, 4]]
result = model.predict(new_student)[0]

categories = {
    2: "High Performing",
    1: "Average Performing",
    0: "At Risk"
}

print("New Student:", categories[result])

... Actual      : [0, 2]
Predicted      : [0 2]
Accuracy       : 100.0 %
New Student:   High Performing
```

23. Advantages

1. Identifies students who may require academic support.
2. Provides early academic-risk prediction.
3. Uses multiple student-related factors.
4. Decision Tree results are relatively easy to interpret.
5. Pandas provides efficient data analysis.
6. The system can support academic decision-making.

24. Limitations

1. Prediction quality depends on the quality and size of the dataset.
2. Student performance can be influenced by factors not present in the dataset.
3. A model trained on one institution's data may not generalize perfectly to another institution.

4. Predictions should support, not replace, teacher or academic decision-making.

25. Conclusion

The proposed Intelligent Student Academic Performance and At-Risk Prediction System uses machine learning to analyse student academic information and classify students into High Performing, Average Performing, and At Risk categories.

The system applies data preprocessing, Pandas-based analysis, feature selection, and Decision Tree classification. Model evaluation using accuracy, precision, recall, F1-score, and confusion matrix helps determine the effectiveness of the developed system.

The system can help educational institutions identify students who may need additional academic support and enable timely intervention.